10) | Lecture 20 | MST & its solution using Kruskal's Algorithm |

15 September 2026 12:05

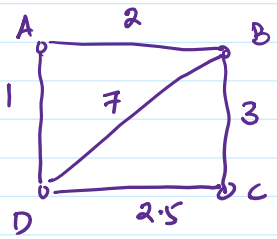
$G = (V, E)$ is a undirected graph

Spanning Tree (i) should include all vertices in G

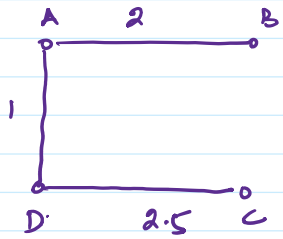
(ii) connected subgraph without cycles

Minimum Spanning Tree : Spanning tree of min weight. Every edge has some non -ve weight

$G = (V, E)$, $W : E \to \mathbb{R}^+$



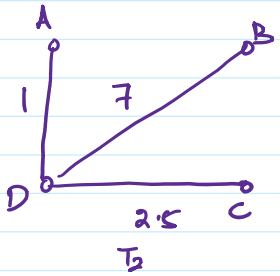
$G = (V, E)$



T_1

$Wt(T_1) = 5.5$

MST



T_2

$Wt(T_2) = 10.5$

Kruskal's Algorithm Input $G = (V, E)$ $|V| = n$

(1) Sort edges based on order of weights

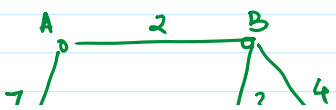
$e_1, e_2 \ldots e_m$ s.t $W(e_i) \leq W(e_{i+1})$ $\forall\, i \in [m]$

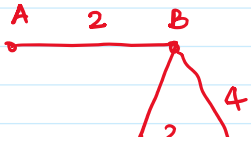
$T \leftarrow \phi$ (MST) [T := set of edges]

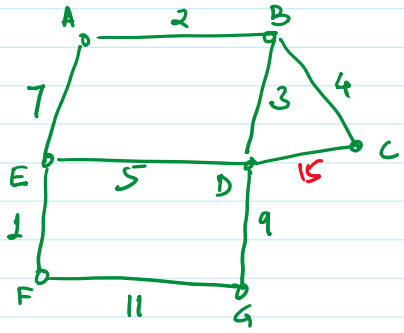
(2) for $i \in 1$ to m and while (# edges(T) $\leq n-1$) do

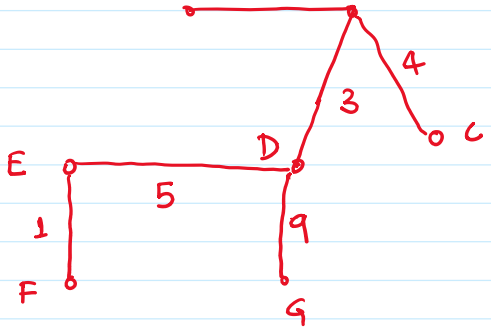
[if $e_i \cup T$ is a tree then (2.1) e_i

$T \leftarrow T \cup \{e_i\}$ (2.2)









$G = (V, E)$

Proof of correctness

Assumption: All edge weight are distinct. let T_{OPT} gives the OPT MST.

Kruskals	g_1	$<$	g_2	$<$	g_3	$\cdots$ g_i $\cdots$	$<$	$g_{n-1} := T_K$
	$\|\|$		$\|\|$		$\|\|$	$\neq$		
T_{opt}	f_1	$<$	f_2	$<$	f_3	$\cdots$ f_i $\cdots$	$<$	$f_{n-1} := T_{OPT}$

g_i := weight of ith edge in kruskals also

f_i := " " " OPT also

Assume that $T_K \neq T_{OPT}$

Suppose T_{OPT} and T_K differ first time at i^{th} location. $g_i \neq f_i$

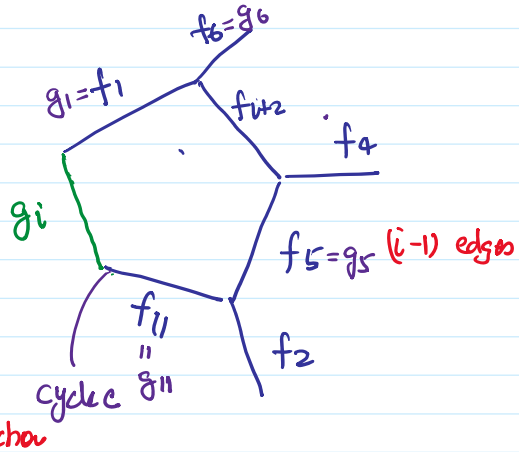
Case 1 $g_i < f_i$

add g_i to T_{OPT} $\{T_{OPT} \setminus f_i\} \cup \{g_i\} := \tilde{T}_{OPT}$

- If there is no cycle formed after adding g_i. then we got a tree $\tilde{T}_{OPT}$ whose weight is smaller than T_{OPT}. Contradiction

$\Rightarrow$ Inclusion of g_i in T_{OPT} create a cycle: $\tilde{T}_{OPT}$

[So far we have added $(i-1)$ edges.]



Suppose we denote cycle by C. let the edges are

f_1, f_3, f_5, f_{11}.

But they are identical to g_1, g_3, g_5, g_{11}

$\Rightarrow$ Inclusion of g_i will also form cycle in kruskal's also. Contradiction

→ Inclusion of g_i will also form cycle in cycle g_{11}

kruskal's algo. Contradiction

Case 2 $f_i < g_i$ (i-1)

f_i is distinct from $g_1 -- g_{i-1}$

Why kruskal's algo didnot pick f_i as $f_i < g_i$.

The only reason is inclusion of f_i in $\{g_1 -- g_{i-1}\} \cup \{f_i\}$ form a cycle.

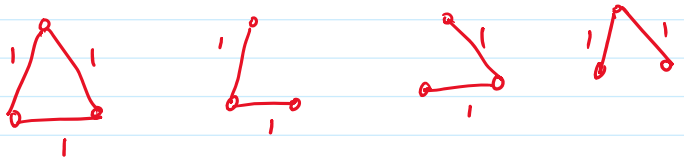
But $f_i \cup \{g_1 -- g_{i-1}\} \equiv f_i \cup \{f_1 -- f_{i-1}\}$

$\Rightarrow$ Inclusion of f_i will also form cycle in OPT algo. Contradiction

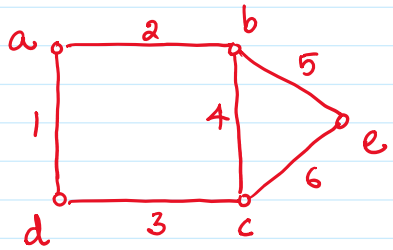
Is the MST Unique?

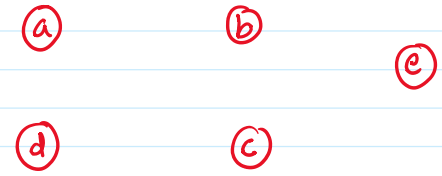
Yes if edges are of distinct weight

No. otherwise



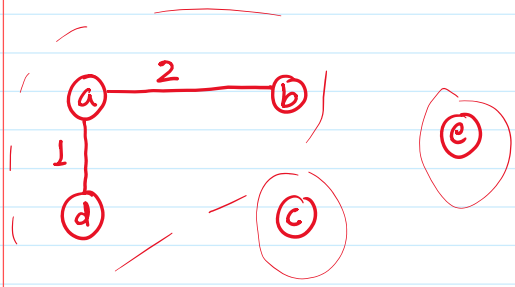
Ex

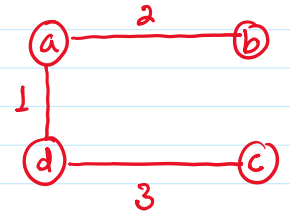




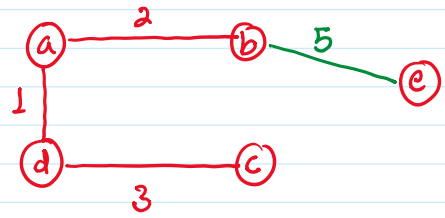
# connect components	Equivalent Representation
5	{a} {b} {c} {d} {e}
4	{a, d} {b} {c} {e}
3	{a, d, b} {c} {e}
2	{a, d, b, c} {e}
1	{a, b, c, d, e}







(b, c)
(b, e)



Adding edge: Taking union of two sets corresponding to connected components

Checking cycle: Checking if both end pt of an edge are in same set

Kruskal's Algorithm Input $G = (V, E)$ $|V| = n-1$

① $T \leftarrow \phi$

② $S = \{v_1\}, \{v_2\} \ldots \{v_n\}$ [* Singleton set of vertices

③ Sort edges based on order of weights
$e_1, e_2 \ldots e_m$ s.t $w(e_i) \leq w(e_{i+1}) \quad \forall i \in [m]$ $O(m \log m)$

④ for $i \leftarrow 1$ to m and while (# edges(T) $\leq n-1$)

do {
$e_i = (u, v)$
if (Find(u) $\neq$ Find(v)) [Find(u) = o/p identifier of set in which u belong

then
$T \leftarrow T \cup \{e_i\}$
Union (Find(u), Find(v))
}

Find() $\leq 2m$ F Complexity of Find

Union() $= n-1$ U " " Union

Overall time complexity $= O(m \log m + n \cdot U + mF)$

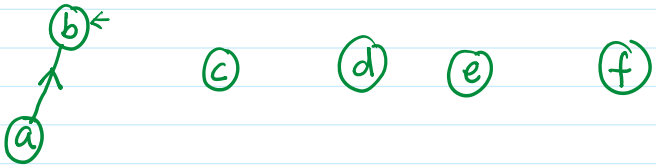
Naive Approach Use linkedlist Union O(1)
Find() O(n)
Not acceptable

u = {a} {b} {c} {d} {e} {f}

(a) (b) (c) (d) (e) (f) a | ptr

add edge (a, b)

Union [Find(a), Find(b)]





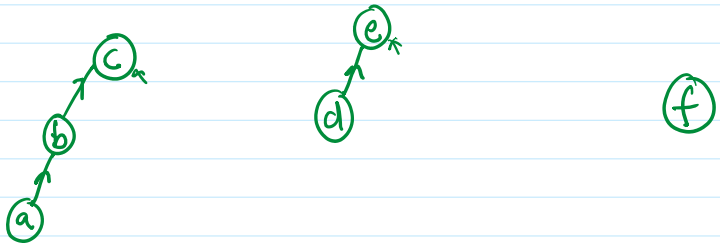
add edge (b,c)

Union (Find(b), Find(c))



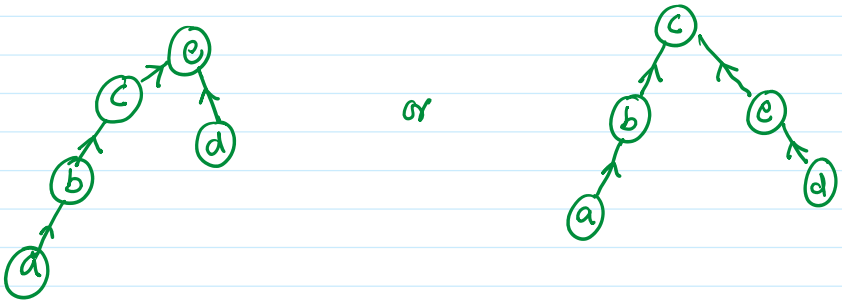
add edge (d, e)

Union (Find(d) = Find(e))?

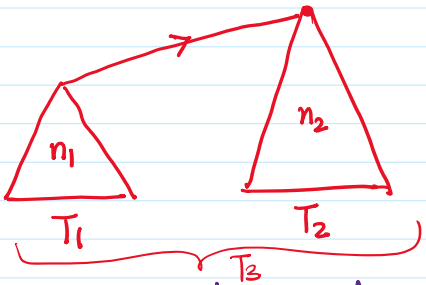


add edge (a, d)

Union (Find(a), Find(d))



Union by rank



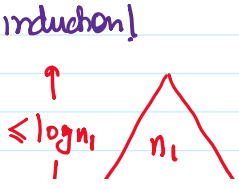
n_i # of nodes in T_i

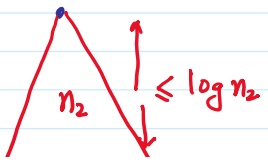
$n_1 < n_2$

Add one variable in root that record # of nodes in the tree

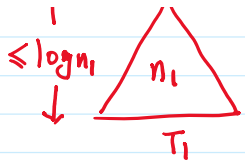
Claim If we follow "union by rank" then the resultant tree with n node has height $< O(\log n)$

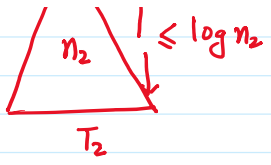
Proof By induction





By assumption





By assumption

Height of recultant tree = $\max \{\log n_2 , (\log n_1) + 1\}$

Need to show

(i) $\log n_2 \leq \log(n_1+n_2)$

True

(ii) $1 + \log n_1 \leq \log(n_1+n_2)$

$n_2 > n_1$

$\frac{n_2}{n_1} > 1$

$1 + \frac{n_2}{n_1} > 2$

$\frac{n_1+n_2}{n_1} > 2$

$\Rightarrow \log\left(\frac{n_1+n_2}{n_1}\right) > 1$

$1 + \log n_1 \leq \log\left(\frac{n_1+n_2}{n_1}\right) + \log n_1$

$= \log(n_1+n_2)$

Union $O(1)$

Find $O(\log n)$

Overall time complexity $O(m \log m + n + m \log n) = O(m \log n)$

as $m < n^2$